Министерство науки и высшего образования РФ
Пермский национальный исследовательский политехнический университет
Электротехнический факультет
Кафедра Информационные технологии и автоматизированные системы

Информатика в приложении к отрасли

Лабораторная работа № 1

Тема: «Методы решения систем линейных алгебраических уравнений»

Выполнил: студент
группы РИС-25-2б
Ильинов Ф. М.
Проверил: профессор кафедры
ИТАС
Шварц К. Г.

г. Пермь – 2026

СОДЕРЖАНИЕ

1 Постановка задачи 4

1.1 Анализ задачи перед ее решением 4

1.2 Задачи работы 5

2 Краткое описание методов 7

2.1 Метод исключения Гаусса с выбором главных элементов 7

2.1.1 Проблема устойчивости и выбор главного элемента 7

2.1.2 Анализ вырождения при нулевом главном элементе 8

2.1.3 Обратный ход метода Гаусса 9

2.2 Метод простой итерации 9

2.2.1 Приведение системы к виду, удобному для итераций 9

2.2.2 Условия сходимости и нормы матриц 10

2.2.3 Обеспечение диагонального преобладания 11

2.2.4 Критерий окончания итерационного процесса 12

3 Исходные данные для конкретных расчетов 13

3.1 Предварительный анализ числовых параметров системы 13

4 Результаты вычислений и программная реализация 15

4.1 Структура программы и ввод данных 15

4.2 Реализация метода Гаусса с выбором главных элементов 16

4.2.1 Программные модули прямого хода 16

4.2.2 Пошаговый протокол выполнения метода Гаусса 19

4.3 Реализация метода простой итерации 21

4.3.1 Программные модули обеспечения сходимости 21

4.3.2 Пошаговый протокол выполнения метода простой итерации . 25

5 Обсуждение результатов 28

5.1 Сравнительный анализ решений и проверка подстановкой 28

5.1.1 Проверка результатов подстановкой в исходную систему 28
5.2 Анализ сходимости итерационного процесса 29
5.3 Сравнение достоинств и недостатков методов 29
5.4 Заключение 30

1 Постановка задачи

Методы решения систем линейных алгебраических уравнений (СЛАУ) можно разделить на точные (прямые) методы, представляющие собой конечный алгоритм для получения решения, и итерационные методы, позволяющие получить решение с заданной точностью путем сходящихся бесконечных процессов. Вследствие неизбежных округлений результаты даже точных методов являются приближенными. При использовании итерационных методов добавляется погрешность самого метода.

Рассматривается система n линейных алгебраических уравнений с n неизвестными:

$$\begin{cases} a_{11}x_1 + a_{12}x_2 + \ldots + a_{1n}x_n = b_1; \\ a_{21}x_1 + a_{22}x_2 + \ldots + a_{2n}x_n = b_2; \\ \ldots\ldots\ldots\ldots\ldots\ldots\ldots\ldots \\ a_{n1}x_1 + a_{n2}x_2 + \ldots + a_{nn}x_n = b_n, \end{cases} \tag{1.1}$$

или в матричной форме записи:

$$Ax = b, \tag{1.2}$$

где A — квадратная матрица коэффициентов системы порядка n; b — вектор свободных членов; x — искомый вектор неизвестных.

1.1 Анализ задачи перед ее решением

Перед непосредственным решением системы требуется провести анализ задачи:

а) **Условие существования и единственности решения**: Система (1.1) имеет единственное решение тогда и только тогда, когда определитель матрицы системы отличен от нуля:

$$\det(A) \neq 0. \tag{1.3}$$

Если $\det(A) = 0$, то на определенном шаге вычислений все коэффициенты при неизвестных в строке обращаются в ноль ($0 \cdot x_1 + \ldots + 0 \cdot x_n$):

– если при этом соответствующий свободный член отличен от нуля ($b_i \neq 0$), получается противоречие $0 = b_i$, и система **не имеет решения**;

– если соответствующий свободный член также равен нулю ($b_i = 0$), получается тождество $0 = 0$, и система имеет **бесконечное множество решений**.

б) **Условия применимости методов**:

– для метода исключения Гаусса: вычисления нельзя производить, если на каком-либо шаге диагональный элемент равен нулю. Кроме того, если элемент на главной диагонали мал, умножение на большие множители приводит к значительным ошибкам при вычитании. Данный недостаток устраняется выбором главного (наибольшего по модулю) элемента;

– для метода простой итерации: метод применим и сходится, если диагональные коэффициенты преобладают по абсолютной величине над остальными коэффициентами строки: $|a_{ii}| > \sum_{j \neq i} |a_{ij}|$. Если в исходной системе хотя бы некоторые диагональные коэффициенты не преобладают, систему с помощью линейного комбинирования уравнений заменяют эквивалентной системой с нужным свойством.

1.2 Задачи работы

В соответствии с индивидуальным заданием (вариант № 11) поставлены задачи:

а) Решить систему линейных уравнений методом Гаусса с выбором главных элементов. Исключить аварийное завершение программы в случае деления на ноль, предусмотрев вывод характера решения (отсутствие решений или бесконечное множество решений);

б) Методом простой итерации (для нечетного варианта) решить систему с точностью $\varepsilon = 10^{-5}$, предварительно приведя ее к виду, удобному для итераций. Для хранения неизвестных x использовать два одномерных массива (на текущей и предыдущей итерациях) и проверять условие сходимости мето-

да через нормы матриц, подчиненные октаэдрической и кубической нормам векторов.

2 Краткое описание методов

2.1 Метод исключения Гаусса с выбором главных элементов

Прямые методы решения систем линейных уравнений позволяют найти точное решение (с точностью до погрешностей машинного округления) за конечное число арифметических операций. Базовым прямым методом является метод последовательного исключения неизвестных Гаусса.

Классический метод Гаусса состоит из двух основных фаз:

а) **Прямой ход**: преобразование исходной системы с помощью эквивалентных элементарных операций над строками расширенной матрицы к эквивалентной системе с верхней треугольной матрицей коэффициентов;

б) **Обратный ход**: последовательное нахождение компонент вектора решения, начиная с последнего уравнения (с одним неизвестным) к первому.

2.1.1 Проблема устойчивости и выбор главного элемента

В стандартной схеме Гаусса на k-м шаге ведущим (опорным) выбирается диагональный элемент $a_{kk}^{(k)}$. Однако такой подход обладает двумя существенными недостатками:

а) если $a_{kk}^{(k)} = 0$, алгоритм аварийно завершается из-за невозможности выполнить деление на ноль;

б) если $|a_{kk}^{(k)}| \ll 1$ (элемент отличен от нуля, но близок к нему), вычисление множителей вида $m_i = -a_{ik}^{(k)}/a_{kk}^{(k)}$ приводит к делению на малую величину. В результате промежуточные коэффициенты строк многократно возрастают, и при их последующем сложении с другими строками происходит потеря значащих цифр из-за вычитания близких чисел (катастрофическая потеря точности).

Для устранения указанных недостатков применяется **метод Гаусса с выбором главного элемента**. В реализованном алгоритме используется схема

выбора главного элемента по всей активной подматрице (полный выбор):

$$|a_{p,q}| = \max_{i,j} |a_{ij}|, \tag{2.1}$$

где p — номер главной строки, q — номер главного столбца, соответствующего неизвестной x_q.

После нахождения главного элемента вычисляются множители для всех остальных строк $i \neq p$:

$$m_i = -\frac{a_{iq}}{a_{pq}}. \tag{2.2}$$

К каждой i-й строке ($i \neq p$) прибавляется p-я строка, умноженная на m_i:

$$a'_{ij} = a_{ij} + m_i a_{pj}, \quad b'_i = b_i + m_i b_p. \tag{2.3}$$

В результате столбец q во всех строках, кроме главной строки p, обращается в ноль. Далее главная строка p и главный столбец q исключаются из рассмотрения, и процедура рекурсивно повторяется для подматрицы порядка $n - 1$.

2.1.2 Анализ вырождения при нулевом главном элементе

В соответствии с замечанием к методу, вычисления нельзя производить, если на каком-либо шаге прямого хода диагональный элемент равен нулю. Перестановкой строк отличный от нуля элемент перемещают на главную диагональ и продолжают вычисления.

Если же оказывается, что все элементы столбца ниже диагонального (и в текущей подматрице) также равны нулю (с машинной точностью $\varepsilon_{\text{mach}} = 10^{-9}$):

$$\max_{i,j} |a_{ij}| \leq \varepsilon_{\text{mach}}, \tag{2.4}$$

то определитель системы равен нулю, и вычисляется характер решения по оставшимся свободным членам b_i:

а) если хотя бы для одной строки свободный член отличен от нуля ($|b_i| > \varepsilon_{\text{mach}}$), то получается противоречие $0 \cdot x_1 + \ldots + 0 \cdot x_n = b_i \neq 0$. В этом случае система **не имеет решения**;

б) если для всех оставшихся строк свободные члены равны нулю ($|b_i| \leq \varepsilon_{\text{mach}}$), получается тождество $0 = 0$. В этом случае система имеет **бесконечное множество решений**.

2.1.3 Обратный ход метода Гаусса

Если вырождения нет, то после завершения прямого хода неизвестные восстанавливаются в порядке, обратном их исключению. Для каждой главной строки p с главным столбцом q:

$$x_q = \frac{1}{a_{pq}} \left(b_p - \sum_{j \neq q} a_{pj} x_j \right), \tag{2.5}$$

где все x_j ($j \neq q$) уже вычислены на предыдущих этапах обратного хода.

При использовании метода Гаусса производится $\frac{1}{3}n(n^2 + 3n - 1)$ операций умножения и деления и $\frac{1}{6}n(n-1)(2n+5)$ вычитаний, то есть всего $\approx \frac{2}{3}n^3$ арифметических операций.

2.2 Метод простой итерации

При большом числе неизвестных (более 10^3) в системе линейных уравнений применение точных методов становится затруднительным. В таких случаях для решения системы удобнее оказываются итерационные методы.

2.2.1 Приведение системы к виду, удобному для итераций

Метод простой итерации состоит в том, что система уравнений $Ax = b$ преобразуется к виду:

$$x = Bx + c, \tag{2.6}$$

которая решается методом последовательных приближений. За нулевое приближение принимается вектор свободных членов $x^{(0)} = c$. Далее вычисляются последовательные приближения по формуле:

$$x^{(k)} = Bx^{(k-1)} + c, \quad k = 1, 2, 3, \ldots \tag{2.7}$$

Если последовательность приближений имеет предел, то этот предел является решением системы.

Преобразовать систему $Ax = b$ к виду $x = Bx + c$ можно, выделив диагональные элементы:

$$x_i = \frac{1}{a_{ii}} \left(b_i - \sum_{j \neq i} a_{ij} x_j \right), \quad i = 1, \ldots, n. \tag{2.8}$$

В матричной форме элементы матрицы B и вектора c определяются выражениями:

$$b_{ij} = \begin{cases} 0, & i = j; \\ -\dfrac{a_{ij}}{a_{ii}}, & i \neq j, \end{cases} \qquad c_i = \frac{b_i}{a_{ii}}. \tag{2.9}$$

2.2.2 Условия сходимости и нормы матриц

Для сходимости метода простой итерации достаточно, чтобы выполнялось условие $\|B\| < 1$ по какой-либо норме матрицы.

Норму матрицы называют согласованной с векторной нормой, если для любого вектора x выполняется условие:

$$\|Bx\| \leq \|B\| \cdot \|x\|. \tag{2.10}$$

Норму матрицы часто вводят через нормы векторов, полагая:

$$\|B\| = \sup_{x \neq 0} \frac{\|Bx\|}{\|x\|}. \tag{2.11}$$

Такую норму матрицы называют матричной нормой, подчиненной векторной норме.

В соответствии с заданием используются две нормы:

а) **Октаэдрическая норма**: для октаэдрической нормы вектора $\|x\|_1 = \sum_{j=1}^{n} |x_j|$ подчиненной матричной нормой является максимум сумм модулей элементов по столбцам:

$$\|B\|_1 = \max_{1 \leq j \leq n} \sum_{i=1}^{n} |b_{ij}|; \tag{2.12}$$

б) **Кубическая норма**: для кубической нормы вектора $\|x\|_\infty = \max_{1\le j\le n}|x_j|$ подчиненной матричной нормой является максимум сумм модулей элементов по строкам:

$$\|B\|_\infty = \max_{1\le i\le n}\sum_{j=1}^{n}|b_{ij}|. \tag{2.13}$$

2.2.3 Обеспечение диагонального преобладания

Матрица A называется матрицей со **строгим диагональным преобладанием**, если для каждой ее строки модуль диагонального элемента строго превосходит сумму модулей всех остальных элементов этой строки:

$$|a_{ii}| > \sum_{j\ne i}|a_{ij}|, \quad \forall i = 1,\dots,n. \tag{2.14}$$

Из соотношений (2.9) и (2.13) непосредственно следует:

$$\|B\|_\infty = \max_{1\le i\le n}\sum_{j\ne i}\frac{|a_{ij}|}{|a_{ii}|}. \tag{2.15}$$

Следовательно, условие строгого диагонального преобладания (2.14) является гарантией того, что кубическая норма матрицы перехода строго меньше единицы:

$$|a_{ii}| > \sum_{j\ne i}|a_{ij}| \iff \sum_{j\ne i}\frac{|a_{ij}|}{|a_{ii}|} < 1 \implies \|B\|_\infty < 1. \tag{2.16}$$

Если исходная матрица системы не удовлетворяет условию диагонального преобладания, ее предварительно преобразуют в эквивалентную систему путем взятия линейных комбинаций строк. В разработанной программе реализован алгоритм перебора коэффициентов линейных комбинаций $w_r \in \{-1, 0, 1\}$:

$$\text{row}'_i = \sum_{r=1}^{n} w_r\,\text{row}_r, \tag{2.17}$$

позволяющий синтезировать новую систему с доминирующей главной диагональю.

2.2.4 Критерий окончания итерационного процесса

В качестве критерия окончания итерационного процесса используется условие малости разности двух последовательных приближений:

$$\|x^{(k)} - x^{(k-1)}\|_\infty = \max_{1 \le i \le n} |x_i^{(k)} - x_i^{(k-1)}| < \varepsilon, \tag{2.18}$$

где $\varepsilon = 10^{-5}$ — заданная точность вычислений. При выполнении этого условия процесс итераций завершается, а полученный вектор $x^{(k)}$ принимается в качестве искомого решения.

3 Исходные данные для конкретных расчетов

В соответствии с индивидуальным заданием для варианта № 11 рассматривается система трех линейных алгебраических уравнений с тремя неизвестными ($n = 3$):

$$\begin{cases} 2{,}7x_1 + 0{,}9x_2 - 1{,}5x_3 = 3{,}5; \\ 4{,}5x_1 - 2{,}8x_2 + 6{,}7x_3 = 2{,}6; \\ 5{,}1x_1 + 3{,}7x_2 - 1{,}4x_3 = -0{,}14. \end{cases} \tag{3.1}$$

Матрица коэффициентов системы A и вектор свободных членов b имеют вид:

$$A = \begin{pmatrix} 2{,}7 & 0{,}9 & -1{,}5 \\ 4{,}5 & -2{,}8 & 6{,}7 \\ 5{,}1 & 3{,}7 & -1{,}4 \end{pmatrix}, \qquad b = \begin{pmatrix} 3{,}5 \\ 2{,}6 \\ -0{,}14 \end{pmatrix}. \tag{3.2}$$

Расширенная матрица системы $\widetilde{A} = (A \,|\, b)$ записывается в виде:

$$\widetilde{A} = \left(\begin{array}{ccc|c} 2{,}7 & 0{,}9 & -1{,}5 & 3{,}5 \\ 4{,}5 & -2{,}8 & 6{,}7 & 2{,}6 \\ 5{,}1 & 3{,}7 & -1{,}4 & -0{,}14 \end{array}\right). \tag{3.3}$$

Заданная точность для итерационного процесса: $\varepsilon = 10^{-5}$.

3.1 Предварительный анализ числовых параметров системы

Перед выполнением численных расчетов проведен аналитический анализ матрицы системы:

а) **Вычисление определителя**:

$$\begin{aligned} \det(A) = 2{,}7 \cdot \big((-2{,}8) \cdot (-1{,}4) - 6{,}7 \cdot 3{,}7\big) - \\ - 0{,}9 \cdot \big(4{,}5 \cdot (-1{,}4) - 6{,}7 \cdot 5{,}1\big) - \\ - 1{,}5 \cdot \big(4{,}5 \cdot 3{,}7 - (-2{,}8) \cdot 5{,}1\big) = -66{,}321. \end{aligned} \tag{3.4}$$

Поскольку $\det(A) = -66{,}321 \neq 0$, матрица невырождена, следовательно, согласно теории решения СЛАУ, система совместна и имеет единственное решение;

б) **Проверка диагонального преобладания**:

– строка 1: $|a_{11}| = 2{,}7$, сумма недиагональных элементов $|a_{12}| + |a_{13}| = 0{,}9 + 1{,}5 = 2{,}4$. Условие $2{,}7 > 2{,}4$ выполняется;

– строка 2: $|a_{22}| = 2{,}8$, сумма недиагональных элементов $|a_{21}| + |a_{23}| = 4{,}5 + 6{,}7 = 11{,}2$. Условие $2{,}8 > 11{,}2$ нарушено;

– строка 3: $|a_{33}| = 1{,}4$, сумма недиагональных элементов $|a_{31}| + |a_{32}| = 5{,}1 + 3{,}7 = 8{,}8$. Условие $1{,}4 > 8{,}8$ нарушено.

Таким образом, исходная система (3.1) **не обладает свойством диагонального преобладания**. Непосредственное построение итерационной матрицы без преобразования приведет к норме $\|B\| > 1$ и расходимости итерационного процесса. Для выполнения задания 2 необходимо эквивалентное линейное комбинирование уравнений.

Исходные числовые данные хранятся во внешнем текстовом файле `11.in` (листинг 1).

```
2.7  0.9  -1.5  3.5
4.5 -2.8   6.7  2.6
5.1  3.7  -1.4 -0.14
```

Листинг 1 – Содержимое входного файла 11.in

4 Результаты вычислений и программная реализация

Программный комплекс для решения поставленных задач разработан на языке программирования C++ и скомпонован в файле `main.cpp`.

4.1 Структура программы и ввод данных

Для универсальности работы с системами произвольной размерности используются стандартные контейнеры библиотеки STL: динамические массивы `std::vector<double>` (тип `vd`) и матрицы `std::vector<std::vector<double»` (тип `vvd`).

Считывание коэффициентов системы осуществляется функцией `readFile` (листинг 2). Функция перенаправляет стандартный поток ввода на целевой файл и выполняет построчный разбор с помощью строкового потока `std::stringstream`, что позволяет автоматически определять размерность системы без жесткого задания констант.

```
vvd readFile(const char *fileName) {
    freopen(fileName, "r", stdin);
    vvd m;

    string fileLine;
    while (getline(cin, fileLine)) {
        stringstream ss(fileLine);

        vd line;
        double num;
        while (ss >> num)
            line.push_back(num);

        m.push_back(line);
    }

    xs.resize(m.size());
    return m;
}
```

Листинг 2 — Функция readFile для универсального считывания матрицы

4.2 Реализация метода Гаусса с выбором главных элементов

4.2.1 Программные модули прямого хода

Поиск главного элемента по всей активной подматрице реализован в функции `findMajor` (листинг 3). Функция сканирует все элементы подматрицы и находит элемент с максимальной абсолютной величиной. В строке 56 выполняется проверка превосходства порога машинного нуля $\varepsilon_{\text{mach}} = 10^{-9}$, что предотвращает деление на ноль при вырожденности системы.

```
47 bool findMajor(const vvd &matrix,
48                 int &majorRow,
49                 int &majorCol,
50                 double &majorVal) {
51     majorVal = 0;
52     bool found = false;
53 
54     for (int i = 0; i < matrix.size(); i++)
55         for (int j = 0; j < matrix.size(); j++)
56             if (abs(matrix[i][j]) > EPS &&
57                 abs(majorVal) < abs(matrix[i][j])) {
58                 majorVal = matrix[i][j];
59                 majorRow = i;
60                 majorCol = j;
61                 found = true;
62             }
63 
64     return found;
65 }
```

Листинг 3 — Функция findMajor для поиска максимального по модулю элемента

Исключение переменной из остальных строк системы выполняется функцией `sumRows` (листинг 4). В строке 75 вычисляется масштабирующий множитель $m_i = -(a_{iq}/a_{pq})$, после чего к каждой строке $i \neq p$ прибавляется главная строка, умноженная на m_i.

```
67 void sumRows(vvd &matrix,
68               const int &majorRow,
69               const int &majorCol,
70               const double &majorVal) {
71     for (int i = 0; i < matrix.size(); i++) {
72         if (i == majorRow)
73             continue;
74 
75         auto m = -(matrix[i][majorCol] / majorVal);
76         for (int j = 0; j < matrix[i].size(); j++)
77             matrix[i][j] += matrix[majorRow][j] * m;
78     }
79 }
```

Листинг 4 — Функция sumRows для исключения главного столбца из строк

Для сокращения размерности подматрицы на каждом шаге алгоритма используется вспомогательная функция `deleteMajor` (листинг 5), которая заполняет нулями главную строку и главный столбец, формируя матрицу для следующего рекурсивного вызова.

```
81 void deleteMajor(vvd &matrix,
82                   const int &majorRow,
83                   const int &majorCol) {
84     vvd newMatrix;
85 
86     for (int i = 0; i < matrix.size(); i++) {
87         vd line;
88         for (int j = 0; j < matrix.size() + 1; j++)
89             if (j == majorCol || i == majorRow)
90                 line.push_back(0);
91             else
92                 line.push_back(matrix[i][j]);
93 
94         newMatrix.push_back(line);
95     }
96 
97     matrix = newMatrix;
98 }
```

Листинг 5 — Функция deleteMajor для обнуления главной строки и столбца

Основная логика решения СЛАУ методом Гаусса сосредоточена в рекурсивной функции `task1` (листинг 6). Если функция `findMajor` не находит ненулевого элемента (строки 106–111), выполняется анализ свободных членов: при наличии ненулевого остатка фиксируется несовместность (возвращается пустой вектор), а при нулевом остатке возвращается признак бесконечного множества решений. Если система совместна, в строках 127–131 выполняется обратный ход Гаусса с вычислением неизвестной x_q.

```
vd task1(vvd &matrix) {
    static int step = 1;
    const int n = matrix.size();

    int majorRow, majorCol;
    double majorVal;
    if (!findMajor(matrix, majorRow, majorCol, majorVal)) {
        for (const auto &row: matrix)
            if (abs(row[n]) > EPS)
                return vd();
        return vd(n, 0);
    }
    cout << "Step: " << step++ << '\n';
    isFree[majorCol] = false;

    sumRows(matrix, majorRow, majorCol, majorVal);
    printMatrix(matrix);

    vvd nextMatrix(matrix);

    deleteMajor(nextMatrix, majorRow, majorCol);
    printMatrix(nextMatrix);

    auto xs = task1(nextMatrix);
    if (xs.empty())
        return xs;

    double free = matrix[majorRow][n];
    for (int j = 0; j < n; j++)
        if (j != majorCol)
            free -= matrix[majorRow][j] * xs[j];
    xs[majorCol] = free / majorVal;

    return xs;
}
```

Листинг 6 — Рекурсивная функция task1 метода Гаусса с обратным ходом

4.2.2 Пошаговый протокол выполнения метода Гаусса

В результате работы функции `task1` над системой варианта № 11 зафиксированы следующие промежуточные этапы:

Шаг 1. Поиск максимума среди всех коэффициентов исходной матрицы:

$$\max_{i,j} |a_{ij}| = |a_{23}| = 6{,}7.$$

Главный элемент расположен во 2-й строке и 3-м столбце (соответствует переменной x_3). Вычисляются множители для строк 1 и 3:

$$m_1 = -\frac{-1{,}5}{6{,}7} \approx 0{,}223881, \qquad m_3 = -\frac{-1{,}4}{6{,}7} \approx 0{,}208955.$$

После сложения строк получаем преобразованную матрицу:

$$\left(\begin{array}{ccc|c} 3{,}70746 & 0{,}273134 & 0 & 4{,}08209 \\ 4{,}5 & -2{,}8 & 6{,}7 & 2{,}6 \\ 6{,}04030 & 3{,}11493 & 0 & 0{,}403284 \end{array}\right).$$

Строка 2 и столбец 3 исключаются из дальнейшего рассмотрения прямого хода.

Шаг 2. Поиск главного элемента среди оставшихся активных коэффициентов (строки 1 и 3, столбцы 1 и 2):

$$\max |a_{ij}| = |a_{31}| = 6{,}04030.$$

Главный элемент расположен в 3-й строке и 1-м столбце (переменная x_1). Множитель для строки 1:

$$m_1 = -\frac{3{,}70746}{6{,}04030} \approx -0{,}613788.$$

После сложения первой строки с третьей, умноженной на m_1, получаем:

$$\left(\begin{array}{ccc|c} 0 & -1{,}63877 & 0 & 3{,}83456 \\ 0 & 0 & 0 & 0 \\ 6{,}04030 & 3{,}11493 & 0 & 0{,}403284 \end{array}\right).$$

Строка 3 и столбец 1 исключаются.

Шаг 3. В активной подматрице остается единственный ненулевой элемент:

$$a_{12} = -1{,}63877 \quad (\text{переменная } x_2), \qquad b_1 = 3{,}83456.$$

Обратный ход:

а) Из последнего уравнения находим x_2:

$$x_2 = \frac{3{,}83456}{-1{,}63877} \approx -2{,}33990;$$

б) Из главной строки шага 2 (строка 3) находим x_1:

$$x_1 = \frac{0{,}403284 - 3{,}11493 \cdot (-2{,}33990)}{6{,}04030} = \frac{7{,}69190}{6{,}04030} \approx 1{,}27343;$$

в) Из главной строки шага 1 (строка 2) находим x_3:

$$x_3 = \frac{2{,}6 - 4{,}5 \cdot 1{,}27343 - (-2{,}8) \cdot (-2{,}33990)}{6{,}7} = \frac{-9{,}68215}{6{,}7} \approx -1{,}44510.$$

Итоговый вектор решения задачи 1 методом Гаусса:

$$x_{\text{Gauss}} = \begin{pmatrix} 1{,}27343 \\ -2{,}33990 \\ -1{,}44510 \end{pmatrix}. \tag{4.1}$$

4.3 Реализация метода простой итерации

4.3.1 Программные модули обеспечения сходимости

Проверка выполнения условия строгого диагонального преобладания реализована в функции `isDominant` (листинг 7).

```
158 bool isDominant(const vvd &m) {
159     int n = m.size();
160     for (int i = 0; i < n; i++) {
161         double off = 0;
162         for (int j = 0; j < n; j++)
163             if (i != j) off += abs(m[i][j]);
164         if (abs(m[i][i]) <= off) return false;
165     }
166     return true;
167 }
```

Листинг 7 — Функция isDominant для проверки диагонального преобладания

Если диагональное преобладание отсутствует, вызывается функция `makeDominant` (листинг 8). Функция выполняет полный перебор целочисленных весов $w_r \in \{-1, 0, 1\}$ линейных комбинаций уравнений, находя такую

эквивалентную систему, в которой диагональные коэффициенты строго превосходят сумму модулей недиагональных элементов.

```
vvd makeDominant(const vvd &m) {
    if (isDominant(m)) return m;
    int n = m.size();
    vvd res = m;

    for (int i = 0; i < n; i++) {
        bool found = false;
        for (int w0 = -1; w0 <= 1 && !found; w0++) {
            for (int w1 = -1; w1 <= 1 && !found; w1++) {
                for (int w2 = -1; w2 <= 1 && !found; w2++) {
                    if (w0 == 0 && w1 == 0 && w2 == 0) continue;
                    vd row(n + 1, 0);
                    int w[3] = {w0, w1, w2};
                    for (int r = 0; r < n; r++)
                        for (int c = 0; c <= n; c++)
                            row[c] += w[r] * m[r][c];

                    double off = 0;
                    for (int j = 0; j < n; j++)
                        if (i != j) off += abs(row[j]);

                    if (abs(row[i]) > off) {
                        res[i] = row;
                        found = true;
                    }
                }
            }
        }
    }
    return res;
}
```

Листинг 8 — Функция makeDominant для синтеза диагонально доминирующей матрицы

Для проверки достаточного условия сходимости метода итераций реализованы функции вычисления норм матрицы `normOct` (октаэдрическая норма, листинг 9) и `normCub` (кубическая норма, листинг 10).

```
136 double normOct(const vvd &B) {
137     double res = 0;
138     for (int j = 0; j < B.size(); j++) {
139         double colSum = 0;
140         for (int i = 0; i < B.size(); i++)
141             colSum += abs(B[i][j]);
142         res = max(res, colSum);
143     }
144     return res;
145 }
```

Листинг 9 — Функция normOct для вычисления октаэдрической нормы матрицы

```
147 double normCub(const vvd &B) {
148     double res = 0;
149     for (int i = 0; i < B.size(); i++) {
150         double rowSum = 0;
151         for (int j = 0; j < B.size(); j++)
152             rowSum += abs(B[i][j]);
153         res = max(res, rowSum);
154     }
155     return res;
156 }
```

Листинг 10 — Функция normCub для вычисления кубической нормы матрицы

Реализация итерационного процесса приведена в функции `task2`. Первая часть функции отвечает за приведение матрицы к виду со строгим диагональным преобладанием, построение матрицы перехода B и вектора c, а также проверку условий сходимости по вычисленным нормам (листинг 11).

```
vd task2(const vvd &matrix) {
    int n = matrix.size();
    cout << "Task 2: Simple Iteration Method\n";

    vvd m = makeDominant(matrix);
    cout << "Transformed matrix (diagonal dominance):\n";
    printMatrix(m);

    vvd B(n, vd(n, 0));
    vd c(n, 0);
    for (int i = 0; i < n; i++) {
        c[i] = m[i][n] / m[i][i];
        for (int j = 0; j < n; j++)
            if (i != j)
                B[i][j] = -m[i][j] / m[i][i];
    }

    cout << "Matrix B:\n";
    printMatrix(B);
    cout << "Vector c:\n";
    for (double val : c) cout << val << ' ';
    cout << "\n\n";

    double nOct = normOct(B);
    double nCub = normCub(B);
    cout << "Octahedral norm ||B||_1   = " << nOct << "\n";
    cout << "Cubic norm      ||B||_inf = " << nCub << "\n";

    if (nOct < 1.0 || nCub < 1.0) {
        cout << "Convergence condition satisfied (norm < 1).\n\n";
    } else {
        cout << "Convergence condition NOT satisfied.\n\n";
    }

```

Листинг 11 — Подготовка матрицы B и проверка сходимости в функции task2

Вторая часть функции `task2` реализует непосредственно итерационный цикл (листинг 12). Согласно требованиям к заданию 2, для хранения неизвестных используются строго два одномерных массива: `x_prev` (вектор значений на предыдущей итерации) и `x_curr` (вектор значений на текущей итерации). Итерационный цикл завершается при достижении условия $\max_i |x_i^{(k)} - x_i^{(k-1)}| < 10^{-5}$.

```
    vd x_prev = c;
    vd x_curr(n);

    int step = 0;
    while (true) {
        step++;
        double maxDiff = 0;

        for (int i = 0; i < n; i++) {
            x_curr[i] = c[i];
            for (int j = 0; j < n; j++)
                x_curr[i] += B[i][j] * x_prev[j];

            maxDiff = max(maxDiff, abs(x_curr[i] - x_prev[i]));
        }

        cout << "Iter " << step << ": ";
        for (double val : x_curr) cout << val << ' ';
        cout << "| diff = " << maxDiff << "\n";

        if (maxDiff < EPS_ITER)
            break;

        x_prev = x_curr;
    }

    cout << "\nResult task 2:\n";
    for (double val : x_curr) cout << val << ' ';
    cout << "\n";

    return x_curr;
}
```

Листинг 12 — Итерационный цикл с двумя одномерными массивами в функции task2

4.3.2 Пошаговый протокол выполнения метода простой итерации

После применения функции `makeDominant` получена эквивалентная система с диагональным преобладанием:

$$\widetilde{A}_{\text{dom}} = \left(\begin{array}{ccc|c} -12{,}3 & -1{,}8 & -3{,}8 & -5{,}96 \\ 2{,}4 & 2{,}8 & 0{,}1 & -3{,}64 \\ 1{,}8 & -3{,}7 & 8{,}2 & -0{,}90 \end{array}\right). \tag{4.2}$$

Строки матрицы (4.2) получены следующими линейными комбинациями исходных уравнений R_1, R_2, R_3:

а) новая строка 1: $-R_1 - R_2 - R_3$, где $|-12{,}3| = 12{,}3 > |-1{,}8| + |-3{,}8| = 5{,}6$;

б) новая строка 2: $-R_1 + R_3$, где $|2{,}8| = 2{,}8 > |2{,}4| + |0{,}1| = 2{,}5$;

в) новая строка 3: $-R_1 + R_2$, где $|8{,}2| = 8{,}2 > |1{,}8| + |-3{,}7| = 5{,}5$.

Выражая диагональные неизвестные, получаем каноническую систему $x = Bx + c$:

$$B = \begin{pmatrix} 0 & -0{,}146341 & -0{,}308943 \\ -0{,}857143 & 0 & -0{,}035714 \\ -0{,}219512 & 0{,}451220 & 0 \end{pmatrix}, \qquad c = \begin{pmatrix} 0{,}484553 \\ -1{,}300000 \\ -0{,}109756 \end{pmatrix}. \tag{4.3}$$

Вычисление подчиненных норм матрицы перехода B:

а) октаэдрическая норма (максимум по столбцам):

$$\|B\|_1 = \max\big(0 + 0{,}857143 + 0{,}219512;\ 0{,}146341 + 0{,}451220;\ 0{,}308943 + 0{,}035714\big) = 1{,}07666;$$

б) кубическая норма (максимум по строкам):

$$\|B\|_\infty = \max\big(0{,}146341 + 0{,}308943;\ 0{,}857143 + 0{,}035714;\ 0{,}219512 + 0{,}451220\big) = 0{,}892857.$$

Так как кубическая норма матрицы $\|B\|_\infty = 0{,}892857 < 1$, **достаточное условие сходимости метода выполнено.**

Динамика приближений на каждом шаге итерационного процесса приведена в таблице 4.1.

Таблица 4.1 — Пошаговый протокол выполнения метода простой итерации

Итерация k	$x_1^{(k)}$	$x_2^{(k)}$	$x_3^{(k)}$	$\|x^{(k)} - x^{(k-1)}\|_\infty$
0 (вектор c)	0,484553	−1,300000	−0,109756	—
1	0,708705	−1,711410	−0,802707	0,692951
2	0,982994	−1,878790	−1,037550	0,274289
3	1,080040	−2,105510	−1,173280	0,226718
4	1,155150	−2,183850	−1,296890	0,123602
5	1,204800	−2,243810	−1,348720	0,059968
6	1,229590	−2,284520	−1,386680	0,040706
7	1,247280	−2,304410	−1,410490	0,023809
8	1,257540	−2,318720	−1,423350	0,014307
9	1,263610	−2,327060	−1,432050	0,008709
10	1,267520	−2,331950	−1,437150	0,005095
11	1,269810	−2,335120	−1,440210	0,003171
12	1,271220	−2,336970	−1,442150	0,001933
13	1,272090	−2,338110	−1,443290	0,001146
14	1,272610	−2,338820	−1,444000	0,000705
15	1,272930	−2,339240	−1,444430	0,000432
16	1,273130	−2,339500	−1,444690	0,000260
17	1,273250	−2,339660	−1,444850	0,000160
18	1,273320	−2,339750	−1,444950	$9{,}72 \cdot 10^{-5}$
19	1,273360	−2,339810	−1,445010	$5{,}92 \cdot 10^{-5}$
20	1,273390	−2,339850	−1,445040	$3{,}61 \cdot 10^{-5}$
21	1,273410	−2,339870	−1,445060	$2{,}20 \cdot 10^{-5}$
22	1,273420	−2,339880	−1,445080	$1{,}34 \cdot 10^{-5}$
23	1,273420	−2,339890	−1,445090	$8{,}17 \cdot 10^{-6}$

На итерации № 23 достигнута разность $\|x^{(23)} - x^{(22)}\|_\infty = 8{,}17 \cdot 10^{-6} < 10^{-5}$, что завершило вычислительный процесс.

Итоговый вектор решения задачи 2 методом простой итерации:

$$x_{\text{Iter}} = \begin{pmatrix} 1{,}27342 \\ -2{,}33989 \\ -1{,}44509 \end{pmatrix}. \tag{4.4}$$

5 Обсуждение результатов

5.1 Сравнительный анализ решений и проверка подстановкой

В результате работы программного комплекса получены численные решения системы уравнений варианта № 11 прямым методом Гаусса с выбором главного элемента и методом простой итерации:

$$x_{\text{Gauss}} = \begin{pmatrix} 1{,}27343 \\ -2{,}33990 \\ -1{,}44510 \end{pmatrix}, \qquad x_{\text{Iter}} = \begin{pmatrix} 1{,}27342 \\ -2{,}33989 \\ -1{,}44509 \end{pmatrix}. \tag{5.1}$$

Максимальное расхождение между результатами методов Гаусса и простой итерации составляет:

$$\begin{aligned} \|x_{\text{Gauss}} - x_{\text{Iter}}\|_\infty &= \\ &= \max\big(|1{,}27343 - 1{,}27342|;\ |-2{,}33990 - (-2{,}33989)|; \\ &\qquad |-1{,}44510 - (-1{,}44509)|\big) = 10^{-5}, \end{aligned} \tag{5.2}$$

что в точности соответствует заданной в условии точности $\varepsilon = 10^{-5}$.

5.1.1 Проверка результатов подстановкой в исходную систему

В соответствии с требованиями к отчету выполнена проверка полученных результатов путем непосредственной подстановки векторов x_{Gauss} и x_{Iter} в исходную систему уравнений (3.1).

Для решения методом Гаусса значения левых частей уравнений составляют:

$$Ax_{\text{Gauss}} = \begin{pmatrix} 2{,}7 \cdot 1{,}27343 + 0{,}9 \cdot (-2{,}33990) - 1{,}5 \cdot (-1{,}44510) \\ 4{,}5 \cdot 1{,}27343 - 2{,}8 \cdot (-2{,}33990) + 6{,}7 \cdot (-1{,}44510) \\ 5{,}1 \cdot 1{,}27343 + 3{,}7 \cdot (-2{,}33990) - 1{,}4 \cdot (-1{,}44510) \end{pmatrix} = \begin{pmatrix} 3{,}500001 \\ 2{,}599985 \\ -0{,}139997 \end{pmatrix}. \tag{5.3}$$

Разности между левыми и правыми частями равны $|(Ax)_1 - b_1| = 1{,}0 \cdot 10^{-6}$, $|(Ax)_2 - b_2| = 1{,}5 \cdot 10^{-5}$, $|(Ax)_3 - b_3| = 3{,}0 \cdot 10^{-6}$.

Для решения методом простой итерации:

$$Ax_{\text{Iter}} = \begin{pmatrix} 2{,}7 \cdot 1{,}27342 + 0{,}9 \cdot (-2{,}33989) - 1{,}5 \cdot (-1{,}44509) \\ 4{,}5 \cdot 1{,}27342 - 2{,}8 \cdot (-2{,}33989) + 6{,}7 \cdot (-1{,}44509) \\ 5{,}1 \cdot 1{,}27342 + 3{,}7 \cdot (-2{,}33989) - 1{,}4 \cdot (-1{,}44509) \end{pmatrix} = \begin{pmatrix} 3{,}499968 \\ 2{,}599979 \\ -0{,}140025 \end{pmatrix}. \tag{5.4}$$

Разности между левыми и правыми частями не превосходят $3{,}2 \cdot 10^{-5}$.

Полученные результаты практически точно совпадают с вектором свободных членов $b = (3{,}5;\ 2{,}6;\ -0{,}14)^T$, что полностью подтверждает правильность найденных решений. Погрешности для метода Гаусса вызваны округлением компонент вектора при выводе до пяти знаков, а для метода итераций согласуются с заданной точностью $\varepsilon = 10^{-5}$.

5.2 Анализ сходимости итерационного процесса

Скорость сходимости метода простой итерации определяется величиной нормы матрицы перехода B. В нашем случае кубическая норма матрицы составила:

$$\|B\|_\infty = 0{,}892857 < 1. \tag{5.5}$$

Поскольку норма $\|B\|_\infty$ меньше единицы, достаточное условие сходимости выполнено. При этом норма достаточно близка к единице ($0{,}893$), что обусловило плавную динамику уменьшения погрешности: для достижения требуемой точности $\varepsilon = 10^{-5}$ потребовалось 23 итерации. На последней итерации разность соседних приближений составила $\|x^{(23)} - x^{(22)}\|_\infty = 8{,}17 \cdot 10^{-6} < 10^{-5}$, что полностью удовлетворяет критерию окончания вычислений.

5.3 Сравнение достоинств и недостатков методов

Сравнительные характеристики прямого метода Гаусса с выбором главных элементов и метода простой итерации сведены в таблицу 5.1.

Таблица 5.1 — Сравнительный анализ методов решения СЛАУ

Характеристика	**Метод Гаусса с выбором главных элементов**	**Метод простой итерации**
Класс метода	Прямой (точный)	Итерационный (приближенный)
Вычислительная сложность	$O(n^3)$ (для $n = 3$: ≈ 18 операций)	$O(k \cdot n^2)$ ($k = 23$ итераций, ≈ 207 операций)
Требования к матрице системы	$\det(A) \neq 0$ (невырожденность)	Наличие нормы $\|B\| < 1$ (диагональное преобладание)
Влияние ошибок машинного округления	Ошибки накапливаются в процессе прямого хода	Метод самоисправляемый; ошибки предыдущих шагов нивелируются
Необходимость начального приближения	Не требуется	Требуется вектор $x^{(0)}$ (обычно выбирается c)
Реакция на вырожденность / деление на 0	Корректно определяет отсутствие решений или бесконечное число решений	При отсутствии диагонального преобладания процесс расходится
Область применения	Плотные матрицы умеренной размерности ($n \leq 10^3$)	Разреженные матрицы сверхбольшой размерности ($n > 10^4$)

5.4 Заключение

В ходе выполнения лабораторной работы № 1 были успешно решены все поставленные задачи:

а) Исходная система уравнений варианта № 11 исследована на совместность и единственность решения ($\det(A) = -66{,}321 \neq 0$, система совместна и определенна);

б) Реализован прямой метод Гаусса с полным выбором главного элемента по матрице, исключающий деление на ноль и аварийное завершение программы;

в) Реализован метод простой итерации с организацией хранения приближений в двух одномерных массивах;

г) Исходная система успешно преобразована в эквивалентную систему со строгим диагональным преобладанием, обеспечив выполнение достаточного условия сходимости $\|B\|_\infty = 0{,}892857 < 1$;

д) Проведена проверка найденных решений подстановкой в исходную систему уравнений (максимальные расхождения с вектором свободных членов составили $1{,}5 \cdot 10^{-5}$ и $3{,}2 \cdot 10^{-5}$ соответственно), что доказывает правильность и высокую точность вычислений.